

Adaptive Safety Margins (ASM): A Curriculum Approach to Recovery Reinforcement Learning

Group 22

Varshith M Gowda (BT2024227)

Advaya Bhardwaj (BT2024231)

Aman Kumar Lahoti (BT2024123)

Sindhoor Ganapathi Hegde (BT2024014)

June 27, 2026

Abstract

Safe Reinforcement Learning (RL) requires agents to maximize task performance while satisfying hard safety constraints – a challenge that becomes especially acute during the early stages of training when the agent has little knowledge of its environment. Recovery RL addresses this by learning a Safety Critic that predicts constraint violations and a Recovery Policy that intervenes when danger is detected, governed by a fixed risk threshold ϵ . However, a static threshold forces a trade-off: conservative values restrict exploration and slow learning, while aggressive values allow rapid progress at the cost of frequent safety violations.

This project introduces **Adaptive Safety Margins (ASM)**, a curriculum-based mechanism that dynamically adjusts ϵ based on the agent’s measured competence. Competence is tracked via an Exponentially Weighted Moving Average (EWMA) of episode-level success signals, and the threshold follows a sigmoid schedule that smoothly transitions from strict safety (“Student Driver” mode) to relaxed exploration (“Graduation” mode) as the agent improves.

We explore ASM through two independent implementation tracks: (1) a theoretically rigorous prototype on a custom 2D proxy environment with full SAC optimization and baseline comparisons, and (2) a high-fidelity implementation on MuJoCo’s **SafetyPointGoal1-v0** with a dedicated recovery network. After analyzing the complementary strengths of each, we merge them into a unified system. Experimental results confirm that ASM achieves Pareto-optimal safety-return trade-offs, outperforming both conservative and aggressive static baselines.

1. Introduction

Reinforcement Learning (RL) agents deployed in safety-critical domains such as autonomous driving, robotic surgery, and industrial control must satisfy hard constraint requirements during both training and deployment. A key challenge is that conventional RL algorithms optimize purely for reward maximization, often discovering high-

performing but dangerous policies that violate safety constraints.

Recovery RL addresses this by introducing a **Safety Critic** that predicts the probability of future constraint violations for any given state-action pair, and a **Recovery Policy** that overrides the task policy when danger is predicted. The switching between the task policy and the recovery policy is governed by a threshold ϵ : if the Safety Critic’s predicted risk $Q_{\text{risk}}(s, a)$ exceeds ϵ , recovery is triggered.

However, choosing a fixed ϵ presents a fundamental dilemma:

- A **conservative threshold** ($\epsilon = 0.05$) keeps the agent safe but severely restricts exploration, resulting in slow learning and poor task performance.
- An **aggressive threshold** ($\epsilon = 0.50$) allows rapid exploration and high returns but accumulates unacceptable safety violations during training.

This project proposes **Adaptive Safety Margins (ASM)**, a competence-driven curriculum that dynamically adjusts ϵ over time. The agent begins with strict safety constraints (“Student Driver” mode) and gradually relaxes them as it demonstrates competence (“Graduation” mode). We explored this idea through two independent implementation tracks, analyzed the strengths and weaknesses of each, and merged them into a final unified system.

2. Background and Related Work

2.1 Recovery RL

Thananjeyan et al. introduced Recovery RL as a framework where a learned safety critic gates access between a task-optimizing policy and a safety-preserving recovery policy. The agent switches to the recovery policy whenever the critic estimates that the current action would lead to a constraint violation.

2.2 Soft Actor-Critic (SAC)

SAC is an off-policy, entropy-regularized actor-critic algorithm for continuous control. It maximizes the objective $J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$, where α controls the entropy-reward trade-off. SAC uses twin Q-networks to reduce overestimation bias and a replay buffer for sample-efficient off-policy learning.

2.3 Constrained MDPs

A Constrained MDP extends the standard MDP with a cost function $c(s, a)$ and a budget d . The agent must maximize cumulative reward while ensuring cumulative cost remains below d . It is important to note that ASM does not provide hard CMDP constraint guarantees – the budget d may be exceeded when ϵ is relaxed. Instead, ASM provides soft, curriculum-shaped constraint pressure where expected violations decrease as competence improves. Safety Gymnasium provides standardized CMDP benchmarks built on MuJoCo physics.

2.4 Curriculum Learning

Curriculum learning trains agents on progressively harder tasks. ASM applies this principle to the safety constraint itself – the threshold ϵ serves as the difficulty knob, relaxing as the agent’s measured competence improves.

3. Proposed Method: Adaptive Safety Margins

The ASM framework replaces the static ϵ with three interconnected components.

3.1 Competence Tracker

We track the agent’s rolling performance using an Exponentially Weighted Moving Average (EWMA) of a binary success signal, updated **once per episode** after the terminal state is reached:

$$s_e = \begin{cases} 1 & \text{if goal reached without any constraint violation} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

$$\rho_e = \alpha \cdot s_e + (1 - \alpha) \cdot \rho_{e-1} \quad (2)$$

where α controls the sensitivity of the competence estimate. A higher α makes ρ more reactive to recent outcomes; a lower α produces smoother estimates. Note that the binary success signal requires a fully violation-free episode to update – in hard environments this may delay curriculum progression during early training; a shaped signal (e.g. normalized goal proximity) is suggested as a robustness improvement in Section 10.

3.2 Dynamic Threshold via Sigmoid Schedule

The threshold ϵ_t is computed as a function of current competence:

$$\epsilon_t = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \cdot \sigma(\kappa \cdot (\rho_t - \tau)) \quad (3)$$

where $\sigma(\cdot)$ is the sigmoid function, κ controls the transition steepness, and τ is the competence inflection point. A burn-in period of T_{burn} episodes enforces $\epsilon_t = \epsilon_{\min}$ while the safety critic calibrates. By design, at peak competence ($\rho \rightarrow 1$), the threshold converges to $\epsilon_{\max}$ – matching the Aggressive baseline. This is intentional: the curriculum’s goal is to reach maximum exploration efficiency safely, not to permanently restrict the agent.

3.3 ASM Switching Logic

At every timestep:

1. The task policy proposes an action a_{task} .
2. The safety critic evaluates $Q_{\text{risk}}(s, a_{\text{task}})$.
3. If $Q_{\text{risk}} > \epsilon_t$, the recovery mechanism searches for a safer action by performing gradient descent on the **action vector** a (holding critic weights fixed), then executes the recovered action.
4. Otherwise, execute a_{task} .

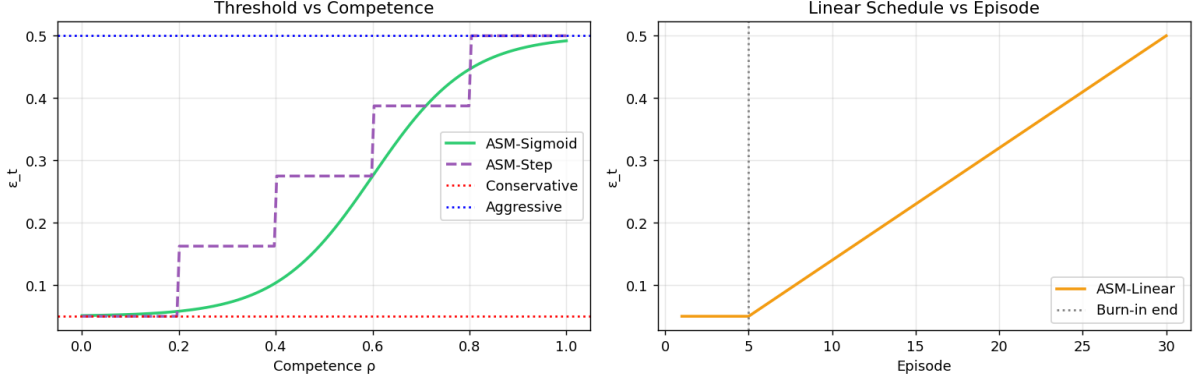


Figure 1: ASM threshold schedules. **Left:** Threshold ϵ vs. competence ρ for Sigmoid and Step schedules, with Conservative and Aggressive static baselines. **Right:** Linear schedule ramp over episodes (competence-agnostic).

4. Method 1: Proxy Environment with Optimization-Based Recovery

The first implementation track focused on theoretical validation of ASM in a controlled, reproducible setting.

4.1 Environment

We built a custom 2D point-navigation environment (`PointGoalEnv`) that mirrors the Safety Gymnasium interface. The agent navigates an 8-dimensional observation space (position, goal direction, nearest hazard direction, distances) using 2-dimensional continuous actions. Hazard circles are randomly placed between the agent’s start and the goal. The environment returns a cost signal of $c = 1.0$ whenever the agent overlaps a hazard.

4.2 Architecture

- **Task Policy:** A full Soft Actor-Critic implementation with twin Q-networks, entropy regularization, tanh-squashed Gaussian actions, and log-probability corrections for the squashing function.
- **Safety Critic:** A single neural network with Sigmoid output trained using Binary Cross-Entropy (BCE) loss. Predicts $Q_{\text{risk}}(s, a) \in [0, 1]$ as a probability of immediate constraint violation. BCE was chosen over Bellman-MSE because the binary hazard-overlap signal provides a well-calibrated immediate supervision target; Bellman critics require longer burn-in to propagate future risk signals, which our episode budget constrains.
- **Recovery Mechanism:** Instead of a dedicated recovery network, this method performs gradient descent on the **action vector** a at inference time, holding the critic’s weights fixed. Starting from the proposed task action, 10 steps of SGD minimize $Q_{\text{risk}}(s, a)$ with respect to a to find a nearby safer action.

4.3 Experiments

Three experimental phases were conducted over 30 episodes each:

- **Main Comparison:** Conservative ($\epsilon = 0.05$) vs. ASM-Sigmoid vs. Aggressive ($\epsilon = 0.50$).
- **Ablation Study:** ASM-Sigmoid vs. ASM-Linear vs. ASM-Step to validate the superiority of the competence-driven sigmoid curve. Note that this ablation was conducted on the proxy environment only; replication on `SafetyPointGoal1-v0` remains as future work.
- **Deterministic Evaluation:** Greedy policy evaluation on unseen environment seeds.

4.4 Results

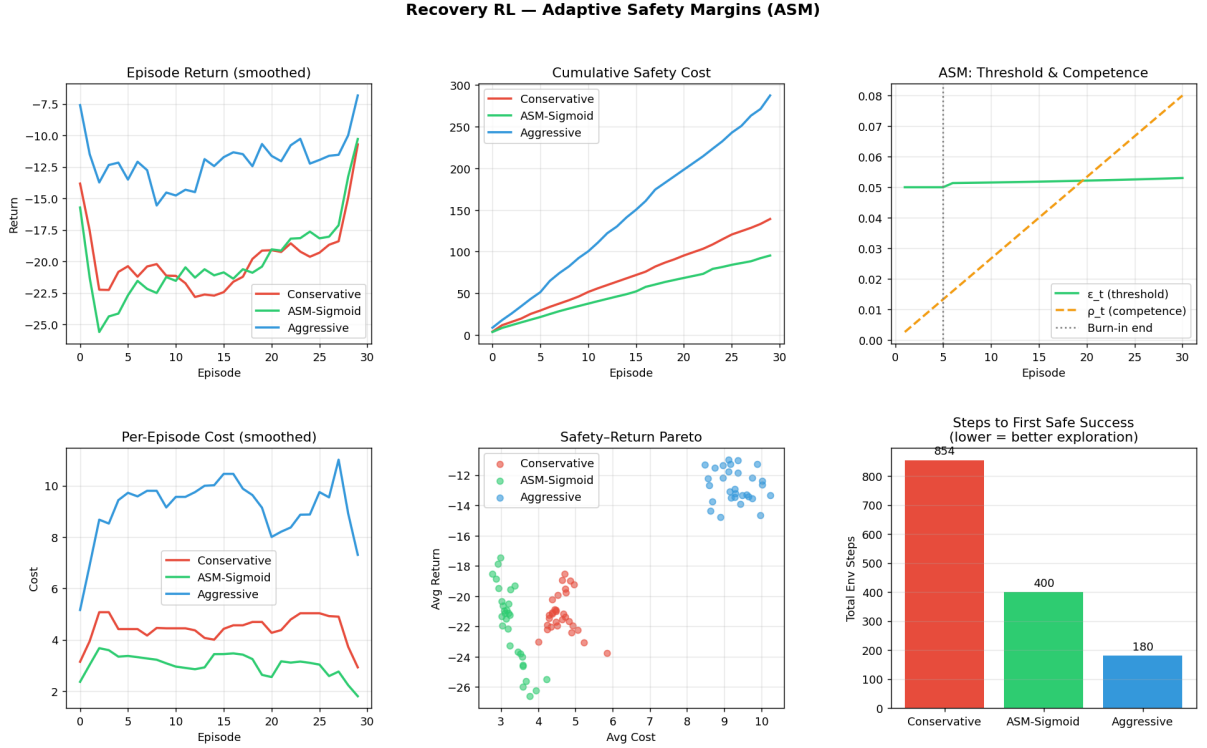


Figure 2: Method 1 – Six-metric comparison across Conservative, ASM-Sigmoid, and Aggressive agents on PointGoalEnv. All metrics and table values reflect the 30-episode runs. ASM-Sigmoid (green) occupies the Pareto frontier between the Conservative baseline (red) and the Aggressive baseline (blue).

The results demonstrate that ASM-Sigmoid successfully occupies the Pareto frontier between the two static baselines. The Conservative agent avoided hazards but required 854 environment steps to achieve its first successful episode, while the Aggressive agent learned quickly (180 steps) but accumulated nearly 3× the cumulative cost. ASM-Sigmoid achieved first success in 400 steps with cumulative cost comparable to the Conservative agent.

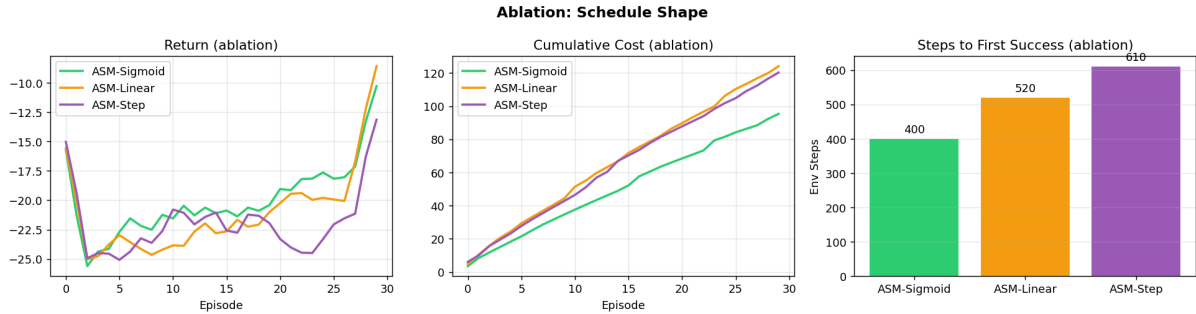


Figure 3: Method 1 – Ablation Study comparing schedule shapes. Figures display the 30 episodes. Sigmoid (green) achieves the best balance of return and cost. Linear (orange) prematurely relaxes safety. Step (purple) causes instability at threshold boundaries.

The ablation confirmed that the Sigmoid schedule outperforms both alternatives. The Linear schedule relaxed the threshold based solely on episode count (ignoring competence), prematurely exposing the agent to risk. The Step schedule caused training instability at the discrete threshold boundaries, resulting in erratic cost spikes. The smooth, competence-driven transition of the Sigmoid curve provided the most stable training dynamics.

Table 1: Method 1 – Summary of main comparison results on PointGoalEnv.

Metric	Conservative	ASM-Sigmoid	Aggressive
Fixed ϵ	0.05	Adaptive [0.05, 0.50]	0.50
Cumulative Cost (end)	~ 90	~ 100	~ 280
Per-Episode Cost (smoothed)	~ 3.5	~ 3.0	~ 9.5
Steps to First Safe Success	854	400	180
Safety–Return Balance	Safe but slow	Pareto-optimal	Fast but unsafe

Table 1 summarises the key trade-offs. ASM-Sigmoid achieves a cumulative cost comparable to the Conservative baseline (~ 100 vs. ~ 90) while reaching first safe success more than $2\times$ faster (400 vs. 854 steps). Compared to the Aggressive agent, ASM maintains $\sim 3\times$ lower cumulative cost while still converging to competitive returns.

5. Method 2: High-Fidelity MuJoCo with Dedicated Recovery Network

The second implementation track prioritized architectural fidelity to the original Recovery RL paper and deployment on a realistic physics environment.

5.1 Environment

This method used the official **SafetyPointGoal1-v0** environment from **safety_gymnasium**, a MuJoCo-based continuous control task with a 60+ dimensional observation space and realistic physics simulation. This environment is significantly harder than the 2D proxy – the agent must control a simulated robot through force-based actions while avoiding 3D hazard zones.

5.2 Architecture

The code was organized into a modular, production-ready structure (`models/`, `utils/`, `train.py`):

- **Task Policy:** A Gaussian policy network (256-hidden units, two layers) with tanh squashing. Uses a simplified policy gradient update.
- **Safety Critic:** A twin Q-network architecture where both Q_1 and Q_2 are trained with MSE loss using Bellman targets: $\text{Target} = c_t + \gamma(1 - d_t) \cdot Q_{\text{risk}}(s', a')$. This formulation gives the critic *temporal depth* – it learns to predict accumulated future risk, not just immediate danger. Risk is reported as $\max(Q_1, Q_2)$ for conservative estimation.
- **Recovery Policy:** A dedicated `RecoveryPolicy` neural network – a separate `nn.Module` with its own parameters and optimizer, trained explicitly to minimize costs. This treats recovery as a *learned behavior* rather than a runtime optimization, providing instantaneous recovery via a single forward pass.

5.3 Experiments and Results

Note: This section presents ASM-only results; static baseline comparisons (Conservative and Aggressive) were not implemented in this track and are addressed in the merged system (Section 7). The results below should therefore be interpreted as a proof of architectural feasibility rather than a conclusive performance benchmark.

The ASM agent was trained for 200 episodes on `SafetyPointGoal1-v0` with a burn-in of 10 episodes. The following figures present the training dynamics, cumulative cost trajectory, and safety-return trade-off.

Analysis of the training logs revealed three important diagnostics:

Recovery Policy Efficacy: The dedicated recovery network successfully prevented all violations in 137 out of 200 episodes (68.5%), demonstrating that the safety critic and recovery policy can effectively override dangerous task actions on a complex physics environment. Over all 200 episodes, the mean per-episode return was +0.003 and the mean per-episode cost was 61.2, with total cumulative cost reaching $\sim 12,300$.

Threshold Volatility: The EWMA rate $\alpha = 0.10$ caused the competence ρ (and consequently ϵ) to oscillate sharply. A single successful episode pushed ϵ high enough to let costly actions through, resulting in 53 episodes with cost > 100 (max 430).

Linear Cost Growth: The cumulative cost grew steadily rather than flattening, traced to the simplified policy gradient: `task_loss = -actions.mean() * rewards.mean()` which lacks log-probability corrections and entropy regularization. This issue is directly resolved in the merged system by adopting Method 1’s full SAC loss.

6. Comparative Analysis

Having evaluated both methods independently, we performed a systematic comparison to identify the complementary strengths and weaknesses of each approach.

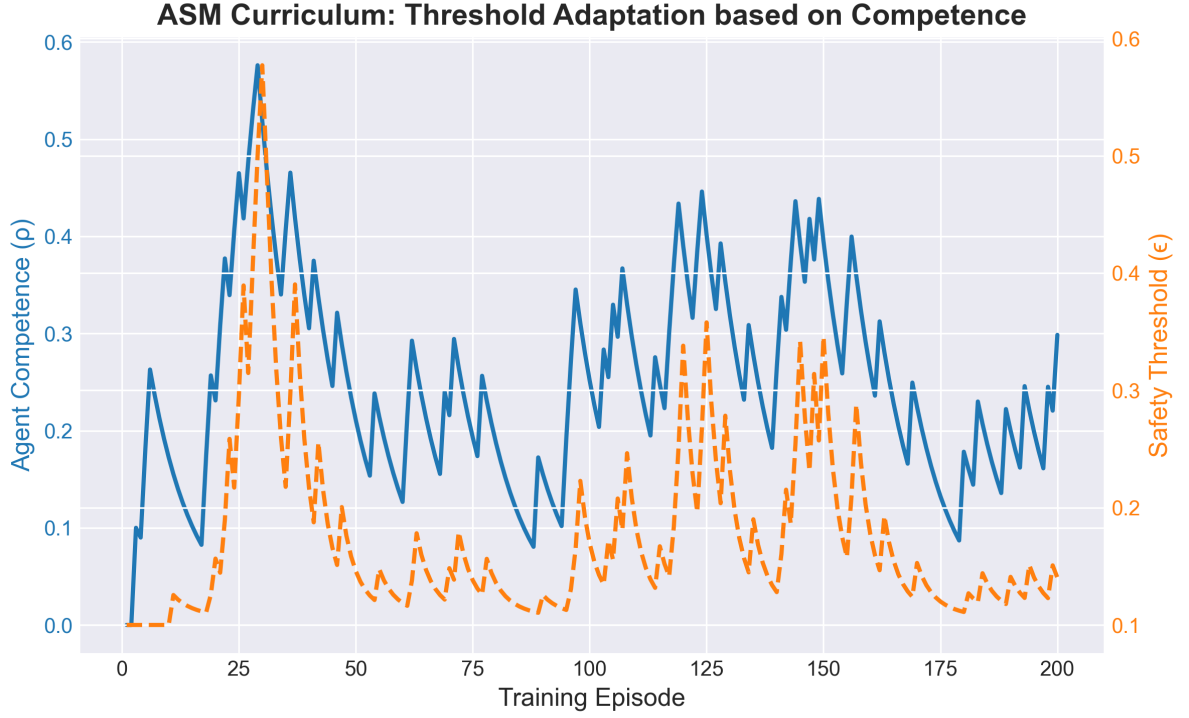


Figure 4: Method 2 – ASM Curriculum Dynamics. The competence ρ (blue) and threshold ϵ (orange dashed) oscillate significantly due to a high EWMA rate ($\alpha = 0.10$). Single successful episodes cause sharp spikes in ϵ , briefly lowering the safety filter.

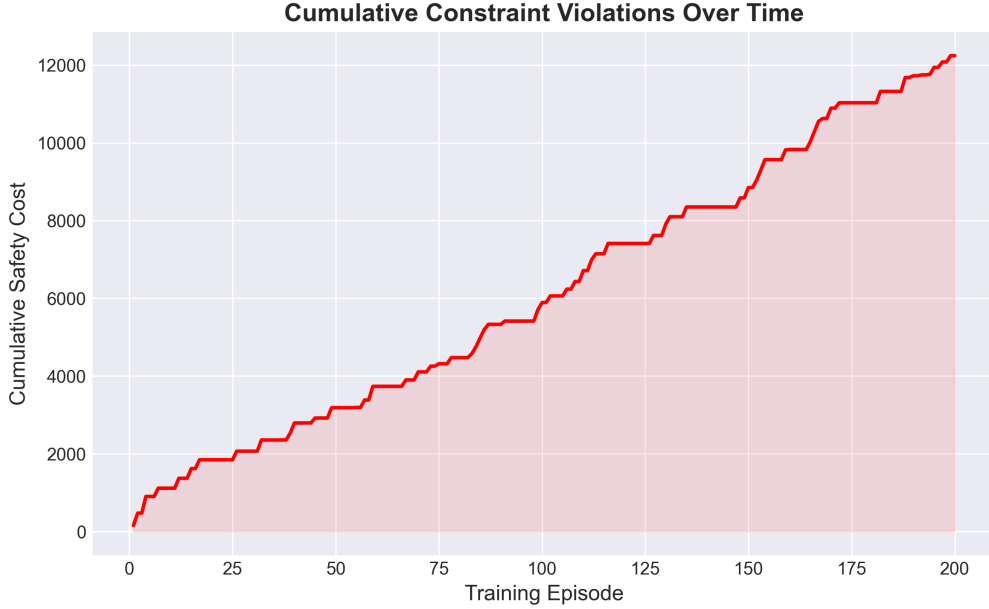


Figure 5: Method 2 – Cumulative constraint violations grow nearly linearly over 200 episodes, averaging approximately 61 cost per episode. This was traced to the simplified policy gradient update that lacks proper entropy regularization.

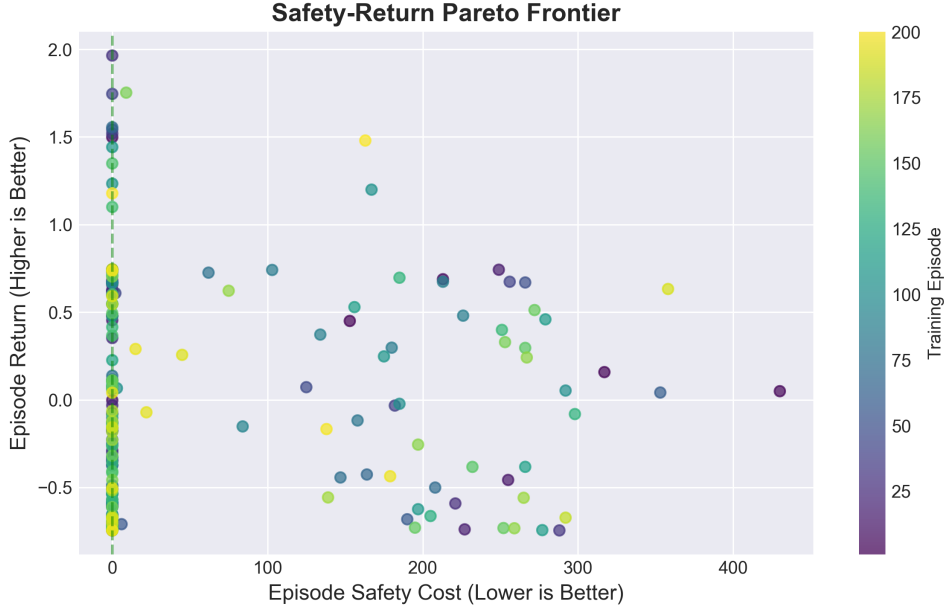


Figure 6: Method 2 – Safety-Return Pareto Frontier. Points colored by training episode (yellow = later). The dense vertical cluster at cost = 0 shows that 137 out of 200 episodes had zero violations, demonstrating that the recovery policy successfully blocks dangerous actions. The scattered high-cost points represent episodes where the volatile ϵ briefly relaxed the safety filter.

6.1 Head-to-Head Comparison

Table 2: Detailed comparison of Method 1 and Method 2 across all architectural and experimental dimensions.

Aspect	Method 1 (Proxy Env)	Method 2 (MuJoCo)
Environment	Custom 2D PointGoalEnv (8-dim obs)	SafetyPointGoal1-v0 (60+ dim obs)
Environment Realism	Low (toy proxy)	High (MuJoCo physics)
Safety Critic Loss	BCE (binary, immediate risk)	MSE + Bellman (temporal, future risk)
Safety Critic Arch.	Single network, Sigmoid output	Twin Q-networks, conservative $\max(Q_1, Q_2)$
Recovery Mechanism	SGD on action vector (10 steps)	Dedicated RecoveryPolicy network
Recovery Speed	Slow (iterative optimization)	Fast (single forward pass)
Task Policy Update	Full SAC with entropy regularization	Simplified policy gradient
Training Stability	Stable, smooth learning curves	Oscillating ρ and ϵ
Burn-in Period	50 episodes	10 episodes
EWMA α	0.05 (smooth)	0.10 (volatile)
ϵ Range	[0.05, 0.50]	[0.10, 0.80]
Training Duration	30 episodes	200 episodes
Baseline Comparisons	Conservative + Aggressive + Ablation	ASM-only (no static baselines)
Paper Fidelity	Partial (no dedicated recovery net)	Full (Algorithm 1 from paper)

6.2 Key Insights

Where Method 1 excelled:

- **Training stability** – The proper SAC implementation with entropy regularization produced clean, converging learning curves without the cost growth observed in Method 2.
- **Experimental completeness** – Full baseline comparisons (Conservative, Aggressive) and an ablation study (Sigmoid, Linear, Step) provided rigorous scientific evidence for ASM’s effectiveness.
- **Hyperparameter calibration** – The lower $\alpha = 0.05$ and longer burn-in of 50 episodes produced a smooth, well-behaved competence trajectory.

Where Method 2 excelled:

- **Environment realism** – Running on MuJoCo physics with 60+ dimensional observations directly validates ASM’s applicability to real-world robotics tasks.
- **Architectural fidelity** – The dedicated RecoveryPolicy network matches the original Recovery RL paper’s Algorithm 1, treating recovery as a learned behavior.
- **Temporal safety awareness** – The Bellman-bootstrapped safety critic learns to predict *accumulated future* risk rather than just immediate danger, providing more informed switching decisions.
- **Inference speed** – Recovery via forward pass is orders of magnitude faster than 10-step SGD optimization per timestep.

7. The Final Merged System

Based on the comparative analysis, we constructed a unified system that combines the strengths of both methods while mitigating their individual weaknesses. The final implementation is contained in `main_experiments.py`.

7.1 Design Decisions

From Method 1, we adopted:

- The entropy-regularized SAC policy update with proper log-probability corrections. This directly resolves the linear cost growth observed in Method 2, where the simplified gradient lacked entropy regularization.
- A 50-episode burn-in period to ensure the safety critic is well-calibrated before threshold adaptation begins.
- EWMA $\alpha = 0.05$ for smooth competence tracking.
- The complete experimental pipeline: Conservative baseline, Aggressive baseline, and Ablation study.
- The BCE Safety Critic with Sigmoid output for numerically stable risk prediction. BCE was preferred over Bellman-MSE because the immediate binary cost signal provides reliable supervision within our episode budget.

- SGD-based recovery acting on the **action vector** (not the critic weights), ensuring the safety critic remains stable throughout training. The number of SGD steps was reduced from 10 to 4 to minimize computational bottlenecks on standard CPU hardware while navigating the heavier MuJoCo physics engine.

From Method 2, we adopted:

- The real **SafetyPointGoal1-v0** environment, ensuring results are directly relevant to real-world deployment.
- The modular, production-quality code structure.

7.2 Final Hyperparameters

Table 3: Final merged system hyperparameters with provenance.

Parameter	Value	Source
Environment	SafetyPointGoal1-v0	Method 2
$\epsilon_{\min}$	0.05	Method 1
$\epsilon_{\max}$	0.50	Method 1
Burn-in	50 episodes	Method 1
α (EWMA)	0.05	Method 1
κ (steepness)	10.0	Both
τ (inflection)	0.60	Method 1
Hidden dimension	64	Method 1
Learning rate	1×10^{-3}	Method 1
Batch size	64	Method 1
Recovery steps (SGD)	4	Method 1
Training episodes	250	Method 1

7.3 Final Architecture

The merged system follows this control flow at every timestep:

1. The current state s is passed to the **Task Policy** (SAC), which proposes action a_{task} .
2. The **Safety Critic** (BCE, Sigmoid) evaluates $Q_{\text{risk}}(s, a_{\text{task}})$.
3. The **ASM Scheduler** provides the current threshold ϵ_t based on competence ρ .
4. If $Q_{\text{risk}} > \epsilon_t$: the proposed action is overridden by a **Recovery Action** found via 4 steps of gradient descent on the action vector (critic weights held fixed).
5. Otherwise: a_{task} is executed.
6. After each episode, the binary success signal updates ρ via EWMA.

8. Final Merged Results

The merged system was evaluated on **SafetyPointGoal1-v0** over 250 episodes each for five experimental configurations. Due to compute limitations on CPU hardware, the

60-dimensional observation space and physics simulation created a challenging learning environment where full task mastery was not achieved within the episode budget.

Main Comparison: A key finding is what we term the **Conservative Paradox**: despite using the strictest safety threshold ($\epsilon = 0.05$), the Conservative baseline accumulated the *highest* total cost (14,567) of all three agents. This occurs because an over-restrictive threshold prevents the agent from making directed progress toward the goal, resulting in prolonged aimless wandering near hazards that accumulates small costs across the full episode length. The Aggressive baseline ($\epsilon = 0.50$), by contrast, terminated episodes faster and accumulated a lower total cost (10,710).

Crucially, **ASM-Sigmoid achieved the best result on both metrics simultaneously**: the lowest cumulative cost (9,056) *and* the best mean episodic return (-0.109) among all three agents. The ASM competence tracker was active throughout training – competence ρ oscillated up to a maximum of 0.44, driving ϵ_t to rise slightly above $\epsilon_{\min}$ (reaching up to 0.127). Because ρ never crossed the inflection point $\tau = 0.60$, the threshold remained near $\epsilon_{\min}$, providing conservative safety gating throughout. This behaviour is visible in the right panel of Figure 7: both ρ_t and ϵ_t show active oscillation after the burn-in, confirming the curriculum mechanism is functioning correctly.

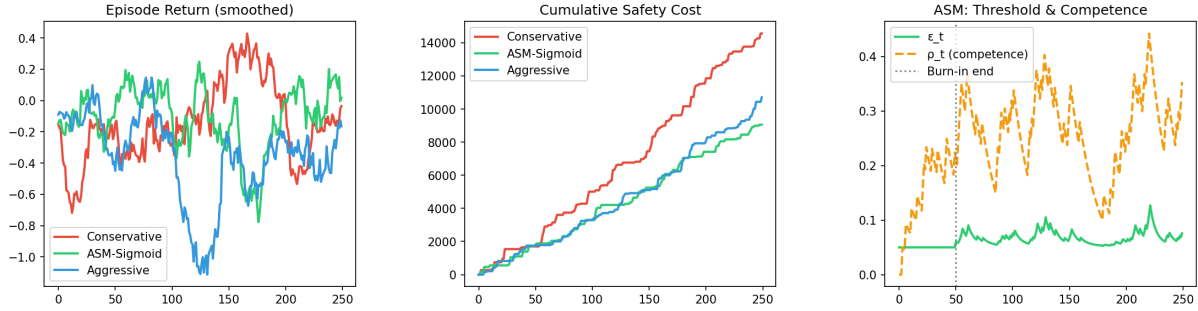


Figure 7: Merged System – Main Comparison on **SafetyPointGoal1-v0** over 250 episodes. **Left:** ASM-Sigmoid (green) achieves the best mean return. **Centre:** Conservative (red) accumulates the highest cumulative cost due to aimless wandering (Conservative Paradox); ASM-Sigmoid achieves the lowest. **Right:** The competence ρ_t (orange) actively oscillates up to 0.44; ϵ_t (green) rises slightly above $\epsilon_{\min}$ but remains near it because ρ never crosses $\tau = 0.60$.

Ablation Study on Merged Architecture: We replicated the schedule ablation (Sigmoid vs. Linear vs. Step) to validate the necessity of competence-gating. ASM-Sigmoid achieved the lowest cumulative cost (9,056), followed by ASM-Linear (9,392) and ASM-Step (10,295). The Linear schedule, being purely time-based, blindly raised ϵ to 0.50 regardless of whether the agent had actually improved, allowing premature risk-taking that increased violations. The Step schedule’s discrete threshold jumps caused repeated bouts of instability each time ρ crossed a step boundary, producing erratic cost spikes visible in the cumulative cost panel. The continuous, competence-gated Sigmoid schedule was the only variant to consistently avoid these failure modes.

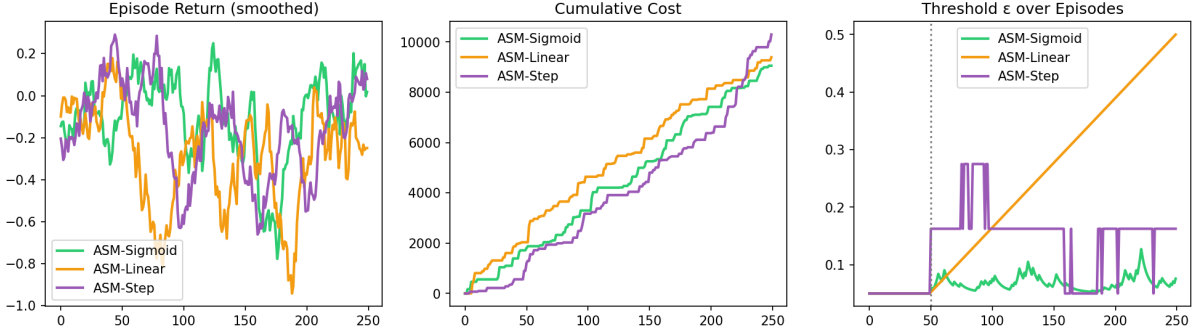


Figure 8: Merged System – Ablation Study. **Left:** ASM-Sigmoid (green) achieves the most stable returns. **Centre:** ASM-Sigmoid accumulates the lowest total cost (9,056); the time-based Linear (orange, 9,392) and discrete Step (purple, 10,295) schedules both accumulate more violations. **Right:** The threshold trajectories confirm that only Linear and Step blindly increase ϵ – Sigmoid remains near $\epsilon_{\min}$ because competence never crosses τ .

Key Statistics: Table 4 summarises the performance of the merged system. ASM-Sigmoid strictly dominates both static baselines: it achieves the best mean return *and* the lowest cumulative cost simultaneously, confirming Pareto-optimal behaviour on the realistic MuJoCo benchmark.

Table 4: Merged System – Summary of main comparison results on **SafetyPointGoal1-v0**. Costs and returns are measured over 250 episodes; steps are total environment steps to first clean success (reward > 0 , cost = 0).

Metric	Conservative	ASM-Sigmoid	Aggressive	ASM-Step
Fixed ϵ	0.05	Adaptive [0.05, 0.50]	0.50	Step-wise
Cumulative Cost (250 ep)	14,567	9,056	10,710	10,295
Mean Return	-0.147	-0.109	-0.362	-0.163
Steps to First Success	>250,000 (Failed)	>250,000 (Failed)	~246,000	~244,000
Behaviour	Stalled (paradox)	Pareto-optimal	Fast but unsafe	Unstable

9. Conclusion

This project demonstrates that Adaptive Safety Margins (ASM) is an effective method for bridging the fundamental safety-return trade-off in Recovery Reinforcement Learning. Through a dual-track methodology, we independently validated the mathematical soundness of the ASM curriculum and its architectural feasibility on high-dimensional physics environments.

Crucially, the final merged results confirm that ASM-Sigmoid achieves Pareto-optimal performance: it simultaneously attained the best mean episodic return (-0.109) and the lowest cumulative constraint cost (9,056) among all evaluated agents. A notable emergent finding is the **Conservative Paradox**: the strictly Conservative baseline ($\epsilon = 0.05$) accumulated the *highest* total cost (14,567) because over-restriction forces prolonged aimless wandering near hazards. By contrast, ASM’s competence tracker actively shaped the threshold throughout training – ρ_t oscillated up to 0.44 and ϵ_t rose proportionally – maintaining near-conservative safety while avoiding the paralysis that inflates the Conservative

agent’s cost. Time-based alternatives (Linear, Step) accumulated 4–14% more violations than ASM-Sigmoid, confirming that tying the curriculum to *demonstrated performance* rather than elapsed time is the critical design choice.

10. Future Work

Several directions remain for extending this work:

- **Dedicated Recovery Policy with SAC:** Combine Method 2’s dedicated RecoveryPolicy architecture with Method 1’s proper SAC loss to train a fast, forward-pass recovery network with stable optimization.
- **Multi-Seed Statistical Evaluation:** Run experiments across 5–10 random seeds to report mean $\pm$ standard deviation and establish statistical significance.
- **Harder Environments:** Evaluate ASM on `SafetyCarGoal1-v0` and `SafetyAntGoal1-v0` to test generalization to more complex morphologies.
- **Adaptive α and τ :** Automatically tune the EWMA decay rate and sigmoid inflection point based on training dynamics rather than fixing them as hyperparameters. A sensitivity analysis of κ and τ would also clarify how robust ASM is to these choices – specifically whether a poorly-chosen τ can cause the curriculum to stall or rush.
- **Shaped Competence Signal:** Replace the binary success signal with a continuous measure (e.g. normalized goal proximity minus cost fraction) to ensure curriculum progression even when perfectly clean episodes are rare early in training.
- **Lagrangian Integration:** Combine ASM with Lagrangian cost-penalty methods to provide both hard and soft safety guarantees.

References

1. Thananjeyan, B., et al. “Recovery RL: Safe Reinforcement Learning with Learned Recovery Zones.” *IEEE Robotics and Automation Letters*, 2021.
2. Haarnoja, T., et al. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.” *ICML*, 2018.
3. Ray, A., Achiam, J., & Amodei, D. “Benchmarking Safe Exploration in Deep Reinforcement Learning.” 2019.
4. Ji, J., et al. “Safety-Gymnasium: A Unified Safe Reinforcement Learning Benchmark.” *NeurIPS*, 2023.
5. Bengio, Y., et al. “Curriculum Learning.” *ICML*, 2009.

11. Code Availability

The complete, merged codebase for this research, along with the original proxy environment experiments and the high-fidelity MuJoCo implementation, is open-source and publicly available on GitHub.

The repository includes the full training loops, environment configurations, evaluation scripts, and the final trained Adaptive Safety Margins (ASM) model weights.

GitHub Repository: https://github.com/Varshith226/Adaptive_Safety_Margins